



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Antonio Manuel Zamora Ayala

N° de Cuenta: 320152896

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 12 de septiembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.1 Crear un cohete espacial con instancias de: pirámides, cilindros, esferas, conos, y cubos.

Dentro de la función *main()* agregué los siguientes bloques de código:

El primer bloque es para dibujar el cilindro y el cono que le dan esa forma característica de un “cohete clásico”:

```
403 // Cuerpo principal del cohete (cilindro)
404 model = glm::mat4(1.0f);
405 model = glm::translate(model, glm::vec3(0.0f, 1.5f, -4.0f));
406 model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
407 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
408 color = glm::vec3(0.8f, 0.8f, 0.8f);
409 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
410 meshList[2] -> RenderMeshGeometry();
411
412 // Punta del cohete (cono)
413 model = glm::mat4(1.0f);
414 model = glm::translate(model, glm::vec3(0.0f, 2.9f, -4.0f));
415 model = glm::scale(model, glm::vec3(0.25f, 0.8f, 0.25f));
416 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
417 color = glm::vec3(1.0f, 0.0f, 0.0f);
418 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
419 meshList[3] -> RenderMeshGeometry();
```

El segundo bloque es dónde dibujo 4 pirámides cuadrangulares para simular las “alas” del cohete:

```
421 // "Ala" izquierda (pirámide cuadrangular)
422 model = glm::mat4(1.0f);
423 model = glm::translate(model, glm::vec3(-0.6f, 0.9f, -4.0f));
424 model = glm::scale(model, glm::vec3(0.4f, 0.8f, 0.4f));
425 model = glm::rotate(model, glm::radians(-30.0f), glm::vec3(0.0f, 0.0f, 1.0f));
426 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
427 color = glm::vec3(1.0f, 0.0f, 0.0f);
428 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
429 meshList[4] -> RenderMesh();
430
431 // "Ala" derecha (pirámide cuadrangular)
432 model = glm::mat4(1.0f);
433 model = glm::translate(model, glm::vec3(0.6f, 0.9f, -4.0f));
434 model = glm::scale(model, glm::vec3(0.4f, 0.8f, 0.4f));
435 model = glm::rotate(model, glm::radians(30.0f), glm::vec3(0.0f, 0.0f, 1.0f));
436 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
437 meshList[4] -> RenderMesh();
```

```

439 // "Ala" trasera (pirámide cuadrangular)
440 model = glm::mat4(1.0f);
441 model = glm::translate(model, glm::vec3(0.0f, 0.9f, -4.6f));
442 model = glm::scale(model, glm::vec3(0.4f, 0.8f, 0.4f));
443 model = glm::rotate(model, glm::radians(30.0f), glm::vec3(1.0f, 0.0f, 0.0f));
444 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
445 meshList[4]->RenderMesh();
446
447 // "Ala" delantera (pirámide cuadrangular)
448 model = glm::mat4(1.0f);
449 model = glm::translate(model, glm::vec3(0.0f, 0.9f, -3.4f));
450 model = glm::scale(model, glm::vec3(0.4f, 0.8f, 0.4f));
451 model = glm::rotate(model, glm::radians(-30.0f), glm::vec3(1.0f, 0.0f, 0.0f));
452 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
453 meshList[4]->RenderMesh();

```

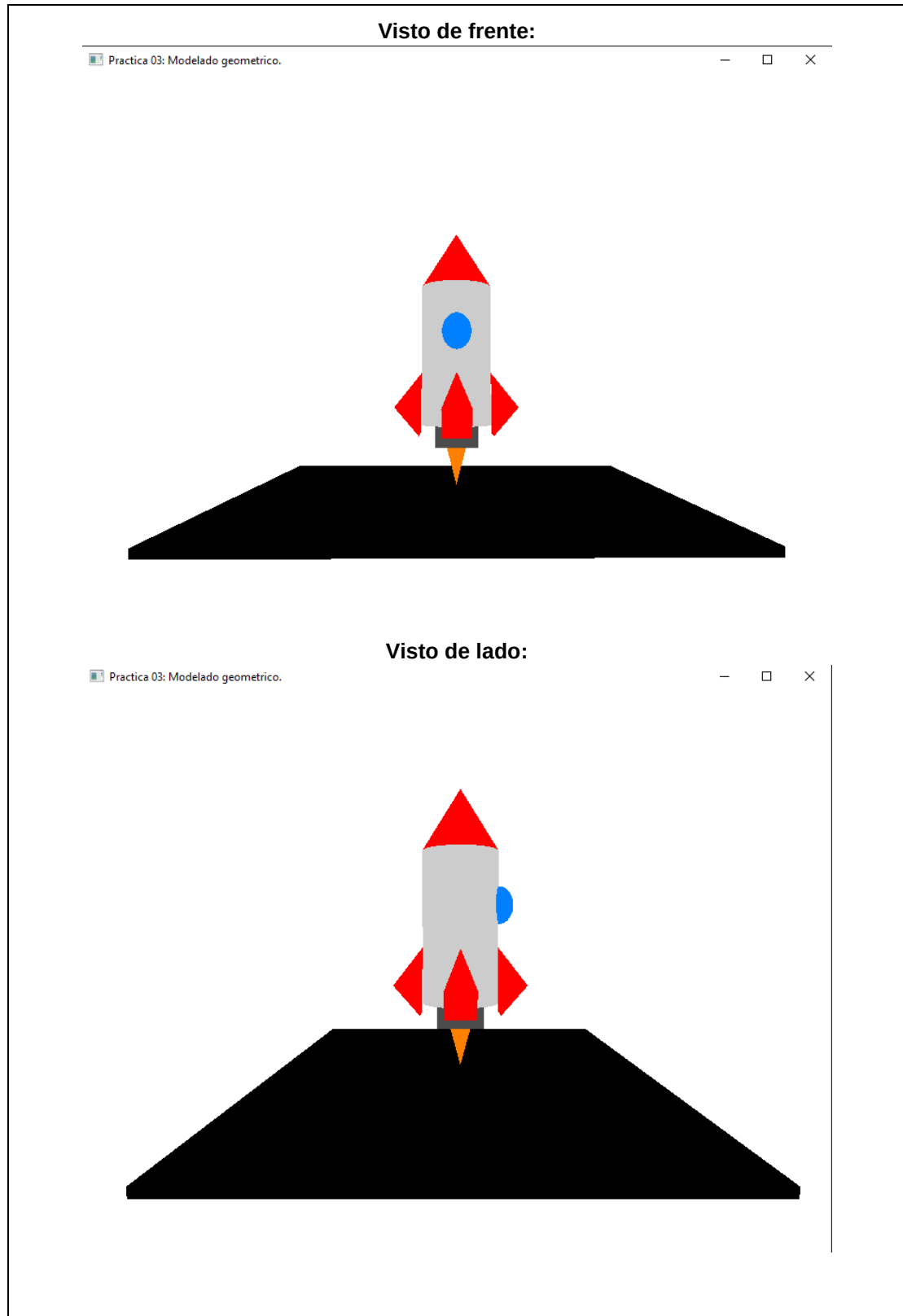
Finalmente, el tercer bloque es dónde dibujo algunos detalles, como una ventana, el motor/propulsor y un poco de fuego:

```

455 // Cristal de la ventana (esfera)
456 model = glm::mat4(1.0f);
457 model = glm::translate(model, glm::vec3(0.0f, 1.8f, -3.5f));
458 model = glm::scale(model, glm::vec3(0.2f, 0.25f, 0.2f));
459 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
460 color = glm::vec3(0.0f, 0.5f, 1.0f);
461 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
462 sp.render();
463
464 // Base del motor (cubo)
465 model = glm::mat4(1.0f);
466 model = glm::translate(model, glm::vec3(0.0f, 0.35f, -4.0f));
467 model = glm::scale(model, glm::vec3(0.6f, 0.3f, 0.6f));
468 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
469 color = glm::vec3(0.3f, 0.3f, 0.3f);
470 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
471 meshList[0]->RenderMesh();
472
473 // Flama del motor (pirámide triangular)
474 model = glm::mat4(1.0f);
475 model = glm::translate(model, glm::vec3(0.0f, 0.05f, -4.0f));
476 model = glm::scale(model, glm::vec3(0.4f, 0.8f, 0.4f));
477 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
478 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
479 color = glm::vec3(1.0f, 0.5f, 0.0f);
480 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
481 meshList[4]->RenderMesh();

```

Adjunto captura de pantalla de mi cohete:





1.2 Generar la figura de unión de 8 pirámides como la que se adjunta. Cada cara de cada pirámide es de un color diferente considerando que al ver una sola pirámide desde todos los ángulos se verían las caras triangulares de color: roja, verde, amarilla, magenta. Y la cara cuadrada se ve de color azul.

Para este ejercicio primero creé un vector para *meshColorList* y también creé una nueva función que creara pirámides cuadrangulares regulares con las caras de distinto color, basándome un poco en lo realizado previamente en la práctica 2:

```
36 | vector<MeshColor*> meshColorList;
```

```

277 void CrearPiramideRegular()
278 {
279     GLfloat vertices_piramide[] = {
280         //Base
281         // X      Y      Z      R      G      B
282         0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 1.0f,
283         0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 1.0f,
284         -0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 1.0f,
285         -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 1.0f,
286         0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 1.0f,
287
288         //Cara frontal
289         -0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f,
290         0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f,
291         0.0f, 0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
292
293         //Cara derecha
294         0.5f, -0.5f, 0.5f, 0.0f, 1.0f, 0.0f,
295         0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 0.0f,
296         0.0f, 0.5f, 0.0f, 0.0f, 1.0f, 0.0f,
297
298         //Cara posterior
299         0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 0.0f,
300         -0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 0.0f,
301         0.0f, 0.5f, 0.0f, 1.0f, 1.0f, 0.0f,
302
303         //Cara izquierda
304         -0.5f, -0.5f, -0.5f, 1.0f, 0.0f, 1.0f,
305         -0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 1.0f,
306         0.0f, 0.5f, 0.0f, 1.0f, 0.0f, 1.0f
307     };
308
309     MeshColor* piramide_color = new MeshColor();
310     piramide_color->CreateMeshColor(vertices_piramide, 18);
311     meshColorList.push_back(piramide_color);
312 }
313

```

Después modifiqué el `glBufferData` de `Mesh.cpp` para que tomara en cuenta que los vértices se componen de 6 valores (XYZ RGB):

```

74 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices[0]) * numOfVertices * 6, vertices, GL_STATIC_DRAW);

```

Posteriormente comencé a armar la figura con 8 pirámides. Los bloques de código modificados para las 8 figuras son:

```

494 //2. Generar la figura de unión de 8 pirámides.
495
496 shaderList[1].useShader();
497 uniformModel = shaderList[1].getModelLocation();
498 uniformProjection = shaderList[1].getProjectLocation();
499 uniformView = shaderList[1].getViewLocation();
500
501 //Pirámide 1
502 model = glm::mat4(1.0);
503 model = glm::translate(model, glm::vec3(0.1f, 0.7f, -2.0f));
504 model = glm::scale(model, glm::vec3(0.5f, 0.2f, 0.5f));
505 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
506 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
507 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
508 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
509 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
510 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
511 meshColorList[0]->RenderMeshColor();
512
513 //Pirámide 2
514 model = glm::mat4(1.0);
515 model = glm::translate(model, glm::vec3(0.1f, 0.5f, -2.0f));
516 model = glm::scale(model, glm::vec3(0.5f, 0.2f, 0.5f));
517 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
518 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
519 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
520 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
521 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
522 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
523 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
524 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
525 meshColorList[0]->RenderMeshColor();
526
527 //Pirámide 3
528 model = glm::mat4(1.0);
529 model = glm::translate(model, glm::vec3(0.1f, 0.3f, -2.0f));
530 model = glm::scale(model, glm::vec3(0.5f, 0.2f, 0.5f));
531 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
532 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
533 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
534 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
535 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
536 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
537 meshColorList[0]->RenderMeshColor();
538
539 //Pirámide 4
540 model = glm::mat4(1.0);
541 model = glm::translate(model, glm::vec3(0.1f, 0.1f, -2.0f));
542 model = glm::scale(model, glm::vec3(0.5f, 0.2f, 0.5f));
543 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
544 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
545 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
546 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
547 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
548 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
549 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
550 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
551 meshColorList[0]->RenderMeshColor();
552

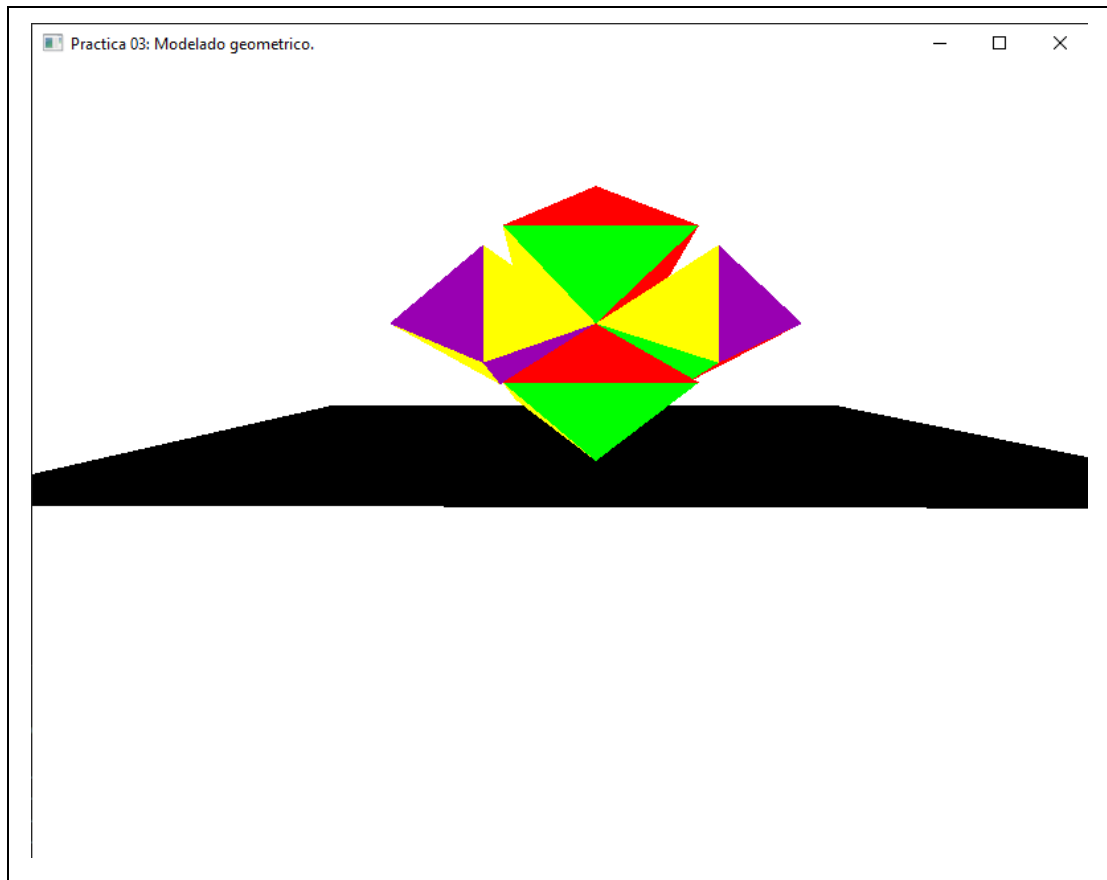
```

```

553 //Pirâmide 5
554 model = glm::mat4(1.0);
555 model = glm::translate(model, glm::vec3(0.25f, 0.4f, -2.0f));
556 model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.5f));
557 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
558 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
559 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
560 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
561 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
562 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
563 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
564 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
565 meshColorList[0] -> RenderMeshColor();
566
567 //Pirâmide 6
568 model = glm::mat4(1.0);
569 model = glm::translate(model, glm::vec3(0.55f, 0.4f, -2.0f));
570 model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.5f));
571 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
572 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
573 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
574 model = glm::rotate(model, glm::radians(270.0f), glm::vec3(0.0f, 0.0f, 1.0f));
575 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
576 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
577 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
578 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
579 meshColorList[0] -> RenderMeshColor();
580
581 //Pirâmide 7
582 model = glm::mat4(1.0);
583 model = glm::translate(model, glm::vec3(-0.35f, 0.4f, -2.0f));
584 model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.5f));
585 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
586 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
587 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
588 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
589 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
590 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
591 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
592 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
593 meshColorList[0] -> RenderMeshColor();
594
595 //Pirâmide 8
596 model = glm::mat4(1.0);
597 model = glm::translate(model, glm::vec3(-0.05f, 0.4f, -2.0f));
598 model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.5f));
599 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
600 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
601 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
602 model = glm::rotate(model, glm::radians(270.0f), glm::vec3(0.0f, 0.0f, 1.0f));
603 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
604 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
605 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
606 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
607 meshColorList[0] -> RenderMeshColor();

```

Adjunto captura de pantalla de la ejecución del programa:



2. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Mi único y mayor problema fue que al momento de hacer el ejercicio 2, la cámara ya no me funcionaba de la misma manera que al hacer el ejercicio 1, es decir, cuando pulsaba AWSO, las pirámides permanecían estáticas, mientras que mi piso si se alejaba/acercaba. Dado lo anterior, sólo podía mover las pirámides haciendo uso de las teclas ERT, por eso mismo, adjunto un video dónde se observa de mejor manera mi planteamiento para construir estas 8 pirámides. También me gustaría aclarar que hice uso de *Microsoft Copilot* y de *Gemini* para que me ayudaran implementar mi función *CrearPiramideRegular()*, porque ya tenía una idea gracias a lo visto en la práctica 2, pero no lograba aterrizarlo en código funcional. Finalmente, también me apoyé de las herramientas mencionadas para tratar de solucionar mi problema con la cámara en el ejercicio 2, pero no tuve éxito.

3. Conclusión:

- a. **Los ejercicios del reporte:** Me agrado mucho el hecho de que ya pudimos tener movilidad con la cámara mediante el mouse y las diversas teclas, asimismo, creo que nos puede servir como breve introducción a lo que haremos en el proyecto final, aunque con una mayor complejidad.
- b. **Comentarios generales:** Considero que fueron ejercicios al nivel de lo que se enseñó en el laboratorio y sirvió para reforzar lo relacionado al modelado geométrico.
- c. **Conclusión.** Esta práctica me sirvió mucho para tener una mejor percepción del espacio en el que trabajamos (3D), ya que mediante el movimiento de la cámara es mucho más sencillo el revisar nuestras figuras y corregir/agregar elementos para las mismas.

Referencias bibliográficas.

OpenGL Tutorial. (n.d.). *Tutorial 4: Un cubo con color*. <https://www.opengl-tutorial.org/es/beginners-tutorials/tutorial-4-a-colored-cube/>

G-Truc Creation. (2020, 5 de enero). *OpenGL Mathematics*. <https://glm.g-truc.net/0.9.9/index.html>